

CustomerPilot V6.1

Production Policies

12 production behavior policies addressing the V6 audit.

No new features - only behavioral clarity + minimal enforcement code.

Each policy: Scenario / Decision Matrix / Behavior / State Machine / Edge Cases / Audit Actions

Table of Contents

1.	Rule Change Migration Policy	Critical
	When owner changes stampsRequired or reward cost, what happens to existing cards?	
2.	VIP Tier Eligibility Policy (Lifetime vs Rolling)	Critical
	Bakery: lifetime Gold is fine. Salon/Gym/Restaurant: inactive Gold members for years is a problem.	
3.	Google Review Reward Lifecycle	Critical
	Customer edits review, deletes it, gives 1-star, or submits duplicate. Bonus stamps lifecycle.	
4.	Birthday Correction Workflow	Critical
	Birthday unknown / wrong / edited / duplicate. What happens to scheduled rewards?	
5.	Campaign Priority Engine	High
	Customer has birthday + win-back + festival + VIP upgrade same week. Don't send 4 WhatsApp messages.	
6.	Reward Stock-Out Behavior	High
	Reward stock = 0. Customer wants to redeem. Alternative? Waitlist? Replace? Don't lose the moment.	
7.	Owner Manual Override Workflow	High
	Owner wants to manually give a customer a reward or bonus stamps. No bill, no redemption flow.	
8.	Duplicate Customer Merge Workflow	High
	Same customer registered twice. Merge without losing history.	
9.	Merchant Ownership Transfer	High
	Bakery sold. New owner. Staff remain. Subscription remains. Data ownership transfer flow.	
10.	Fraud Detection for Fake Bills	Medium
	Cashier creates 5 fake bills for same customer in 10 minutes. Extra stamps. Detection rules.	
11.	AI Explanation Layer for Dashboard Insights	Medium
	Dashboard says 'Repeat Rate dropped 12%'. Merchant asks 'why?'. AI explanation layer answers.	
12.	Daily Summary Scheduling for Different Business Hours	Medium
	Merchant closes shop at 11 PM. Daily summary at 6 PM misses 5 hours. Timezone-aware closing logic.	

1

Rule Change Migration Policy

When owner changes stampsRequired or reward cost, what happens to existing cards?

Scenario

Month 6: Owner Ravi changes 'Cake Lover's Card' from 9 stamps to 6 stamps. Customer A has 5 stamps, Customer B has 6 stamps. Should B instantly get a reward? Should A's progress bar jump? Can Ravi preview before applying? Can he rollback?

Decision Matrix

■ Apply to future cards only

Safest. Existing customers keep old contract. But some customers may feel unfairly locked into a harder rule.

■ Apply to all active cards instantly (current V6 behavior)

Simple. Customer B gets instant reward. But A may feel cheated.

✓ Apply to all active cards + preview + 24h rollback window

Best of both. Owner sees impact preview. Can rollback within 24h if change was a mistake.

RECOMMENDED POLICY

■ Apply from selected future date

Useful for planned promotions. But adds complexity.

Production Behavior

- DEFAULT POLICY: 'Apply to all active cards + preview + 24h rollback window'.
- Owner opens stamp card edit dialog, changes stampsRequired from 9 to 6.
- System shows impact preview BEFORE saving: 'X customers at 6+ stamps will instantly complete. Y customers at 4-5 are now 1-2 away.'
- Owner confirms. Rule applies immediately.
- Customers at 6+ stamps: CustomerStampCard.completed auto-set to true, WhatsApp 'reward_ready' queued.
- Customers at 4-5 stamps: stampsCollected unchanged, but progress bar visually jumps (denominator shrank).
- AuditLog: CARD_EDITED with oldStampsRequired, newStampsRequired, impactedCustomerCount.
- 24-hour rollback window: Owner can click 'Undo rule change'. System reverses — completed cards reset, stampsCollected recalculated. WhatsApp 'apologies, rule change reversed' sent to impacted customers.
- After 24h: change becomes permanent. No rollback possible.

State Machine - States

draft_change | previewed | applied | rolled_back | permanent

Transitions

draft_change -> previewed	// Owner clicks 'Preview impact'
previewed -> applied	// Owner clicks 'Confirm and apply'
applied -> rolled_back	// Owner clicks 'Undo' within 24h
applied -> permanent	// 24h elapse without rollback

Edge Cases

- Reward cost change (1 card to 2 cards): Grandfather existing completed cards for 30 days at old cost.
- Stamp rule change (1 per \$10 to 1 per \$15): Only affects future bills. Past stamps not recalculated.
- Card deactivated: Existing in-progress cards stay active until redeemed or expired.
- Rollback after customer already redeemed: Redemption stays. Owner absorbs cost.

Audit Actions

CARD_EDITED | CARD_RULE_CHANGE_PREVIEWED | CARD_RULE_CHANGE_APPLIED |
CARD_RULE_CHANGE_ROLLED_BACK

Code :

2

VIP Tier Eligibility Policy (Lifetime vs Rolling)

Bakery: lifetime Gold is fine. Salon/Gym/Restaurant: inactive Gold members for years is a problem.

Scenario

Sweet Crumb Bakery: Meera hits Gold and stays Gold forever — even if she stops visiting for 2 years. For a bakery this is OK. But for a Salon where Gold = 30% off, an inactive Gold member is a revenue leak.

Decision Matrix

■ Lifetime VIP (current V6 behavior)

Simplest. Best for low-marginal-cost (bakery, cafe). Worst for high-cost (salon, gym).

■ Rolling 12-month VIP

Fair for high-cost businesses. But harsh — a loyal 5-year customer loses Gold after one quiet year.

■ Rolling 24-month VIP

Balanced. Longer grace period. Slightly more complex.

✓ Merchant setting: choose Lifetime / Rolling-12mo / Rolling-24mo per merchant

Maximum flexibility. Default by business type.

RECOMMENDED POLICY

Production Behavior

- DEFAULT POLICY: 'Merchant setting with business-type default'.
- On signup, merchant picks VIP eligibility mode. Default by business type:
 - - cafe, bakery, retail -> Lifetime (low marginal cost)
 - - salon, gym, restaurant, spa -> Rolling 12-month (high marginal cost)
 - - other -> Rolling 24-month (balanced)
- Cron (weekly Monday 9 AM): for merchants with Rolling mode, recompute rolling-window spend.
- If rolling spend drops below tier threshold: customer is downgraded ONE tier (Platinum -> Gold, Gold -> Silver, Silver -> none).
- Downgrade is NOT instant — 30-day grace + WhatsApp: 'Your Gold status is at risk.'
- If customer visits during grace: status maintained for another 12 months.
- If grace expires: downgrade applied. Achievement badge stays (historical record) but active perks revoked.
- Upgrades remain instant (no grace).

State Machine - States

stable | at_risk | grace_period | downgraded

Transitions

stable -> at_risk	// Rolling spend drops below threshold
at_risk -> grace_period	// 30-day grace starts (WhatsApp sent)
grace_period -> stable	// Customer visits during grace
grace_period -> downgraded	// Grace expires with no visit
downgraded -> stable	// Customer re-qualifies via new spend

Edge Cases

- Merchant switches from Lifetime to Rolling: existing VIPs keep tier for 12 months from switch date.
- Merchant switches from Rolling to Lifetime: all current tiers frozen permanently.
- Customer in grace period redeems a reward: reward proceeds normally (perks still active).
- Customer downgraded mid-redemption-flow: pending_redemptions proceed at old tier cost.

Audit Actions

VIP_TIER_UPGRADED | VIP_TIER_DOWNGRADED | VIP_TIER_AT_RISK |
VIP_CONFIG_CHANGED

Code :

3

Google Review Reward Lifecycle

Customer edits review, deletes it, gives 1-star, or submits duplicate. Bonus stamps lifecycle.

Scenario

Meera submits a 5-star review, gets 2 bonus stamps. Next day she edits it to 3 stars. Or she deletes it entirely. Or her roommate uses same Google account for a 2nd review. What happens to bonus stamps?

Decision Matrix

■ Bonus permanent once awarded

Simplest. But incentivizes review-spam (submit, get stamps, delete, repeat).

■ Claw back on any edit/delete/rating-drop

Strict. But harsh on legitimate edits ('fixed a typo').

✓ Claw back only on DELETE or rating drop below 3 stars. Edits keeping rating ≥ 4 keep bonus.

Balanced. Protects against fraud without punishing honest refinements.

RECOMMENDED POLICY

■ Time-decayed clawback: full within 7 days, 50% within 30 days, 0% after 30 days

Most fair. Accounts for review 'aging'. Complex.

Production Behavior

- DEFAULT POLICY: 'Claw back on DELETE or rating drop below 3 stars. Edits keeping rating ≥ 4 keep bonus.'
- On review submission: bonus stamps awarded immediately. WhatsApp thank-you sent.
- Webhook from Google: if review EDITED and new rating $\geq 4 \rightarrow$ no action. Bonus stays.
- If review EDITED and new rating drops to 1-2 stars $\rightarrow$ claw back bonus stamps. Stamp rows deleted.
- If review DELETED $\rightarrow$ claw back bonus stamps (regardless of original rating).
- If review FLAGGED as fake by Google $\rightarrow$ claw back + customer flagged for fraud review.
- DUPLICATE PREVENTION: one bonus per (customerId, platform) per 90 days.
- Clawback is silent on Customer.lifetimeStamps (historical metric) — only active card balance reversed.

State Machine - States

submitted | bonus_awarded | edited | deleted | clawed_back | flagged_fake

Transitions

submitted $\rightarrow$ bonus_awarded	// POST /api/reviews/submit
bonus_awarded $\rightarrow$ edited	// Google webhook: review edited
edited $\rightarrow$ clawed_back	// New rating < 3 stars $\rightarrow$ clawback
bonus_awarded $\rightarrow$ deleted	// Google webhook: review deleted
deleted $\rightarrow$ clawed_back	// Automatic clawback on delete
bonus_awarded $\rightarrow$ flagged_fake	// Google flags review as fake
flagged_fake $\rightarrow$ clawed_back	// Automatic clawback + fraud review

Edge Cases

- Customer claws back bonus, re-submits within 90 days: no new bonus (duplicate window).
- Clawback would make stampsCollected negative: clamp to 0, log discrepancy.
- Clawback after redemption: redemption stays. Owner absorbs cost.
- Google API outage: webhook delayed by 24h+ -> clawback still proceeds when webhook arrives.

Audit Actions

REVIEW_BONUS_AWARDED | REVIEW_BONUS_CLAWED_BACK | REVIEW_FLAGGED_FAKE |
REVIEW_DUPLICATE_BLOCKED

Code :

4

Birthday Correction Workflow

Birthday unknown / wrong / edited / duplicate. What happens to scheduled rewards?

Scenario

Meera registers without birthday. She visits 3 months later and mentions 'my birthday was last week'. Or: cashier entered wrong birthday (12-03 instead of 12-30). Or: customer has 2 records with different birthdays.

Decision Matrix

■ Birthday immutable once set

Simplest. But punishes data-entry errors.

■ Birthday editable anytime, no questions asked

Customer-flexible. But enables abuse (change to 'today' for free reward).

✓ Birthday editable with manager approval + 30-day cooldown per customer

Balanced. Prevents abuse. Manager verifies.

RECOMMENDED POLICY

■ Birthday editable only within 7 days of original entry

Strict. But what if customer mentions wrong birthday 2 months later?

Production Behavior

- DEFAULT POLICY: 'Birthday editable with manager approval + 30-day cooldown per customer'.
- BIRTHDAY UNKNOWN: Customer.birthday = null. No Birthday record. No reminders. Cashier can add anytime (first set, no cooldown).
- BIRTHDAY FIRST SET: Manager or cashier enters. No approval needed. Birthday record created.
- BIRTHDAY EDIT (already set): Requires MANAGER role. Customer.birthdayLastEditedAt recorded. 30-day cooldown.
- If edit attempted within cooldown: 'Birthday was last edited on X. Wait until Y.' 409 error.
- EDIT AFTER BIRTHDAY WINDOW: If customer didn't redeem, manager can edit, new birthday applies next year.
- EDIT DURING BIRTHDAY WINDOW: If already redeemed this year, edit blocked until next year.
- DUPLICATE BIRTHDAY (merge): Manager picks which birthday to keep. Other discarded.
- WRONG BIRTHDAY (customer reports): Manager verifies (verbal confirmation). Edits. Cooldown applies.
- Suspicious edit (birthday changed to today): system allows but flags for fraud review.

State Machine - States

unknown | set | edit_requested | edited | cooldown

Transitions

```
unknown -> set           // First birthday entry (any staff)
set -> edit_requested     // Manager opens edit dialog
edit_requested -> edited   // Manager confirms (cooldown check passes)
```

```
edited -> cooldown           // 30-day cooldown starts  
cooldown -> set               // 30 days elapse
```

Edge Cases

- Feb 29 (leap year): treated as Feb 28 in non-leap years.
- Customer born in current month: 'birthday' interpreted as upcoming occurrence.
- Manager edits birthday to today (suspicious): flagged for fraud review.
- Bulk import with birthday column: all birthdays set as 'first set' (no cooldown).

Audit Actions

BIRTHDAY_SET | **BIRTHDAY_EDITED** | **BIRTHDAY_EDIT_BLOCKED_COOLDOWN** |
BIRTHDAY_EDIT_SUSPICIOUS | **BIRTHDAY_MERGED**

Code :

5

Campaign Priority Engine

Customer has birthday + win-back + festival + VIP upgrade same week. Don't send 4 WhatsApp messages.

Scenario

Kavya's birthday is next week. She's also at 45 days inactive (win-back). Sweet Crumb running Diwali festival campaign. And she just crossed Rs 10,000 spend (Platinum upgrade). Without priority, Kavya gets 4 WhatsApp in 5 days.

Decision Matrix

■ Send all messages — customer can mute

Simple. But high unsub rate. WhatsApp provider may flag as spam.

■ One message per customer per day, no exceptions

Strict. But important messages (birthday) may get delayed.

✓ Priority engine: 1 message per customer per 48h, highest priority wins

Balanced. Birthday always wins. Lower-priority suppressed or delayed.

RECOMMENDED POLICY

■ Message bundle: combine into one digest

Elegant but complex to draft. 'Happy birthday + Diwali + Platinum' reads weird.

Production Behavior

- DEFAULT POLICY: 'Priority engine: 1 message per customer per 48h, highest priority wins'.
- Priority order (highest to lowest):
 - 1. birthday_reward_ready (highest — 7-day window)
 - 2. birthday_reminder_sent
 - 3. winback_day_60/90 (phone calls, no WhatsApp collision)
 - 4. winback_day_30/45 (WhatsApp)
 - 5. vip_upgrade
 - 6. festival_broadcast
 - 7. review_request
 - 8. stamp_earned (transactional — exempt)
 - 9. redemption_confirm (transactional — exempt)
- TRANSACTIONAL EXEMPTION: stamp_earned, redemption_confirm, referral_bonus always sent immediately.
- MARKETING MESSAGES: subject to 48h suppression.
- ON COLLISION: if new message has HIGHER priority -> new sent, old content appended as P.S.
- If new message has LOWER priority -> suppressed. Sent 48h later if still relevant.
- BUNDLING: if 2+ marketing messages collide within 6h, system bundles into one.

State Machine - States

queued | priority_check | sent | suppressed | bundled | delayed_48h

Transitions

```
queued -> priority_check           // Background worker picks up
priority_check -> sent                 // Transactional OR no collision OR highest priority
priority_check -> suppressed          // Lower priority than existing recent message
priority_check -> bundled             // 2+ marketing messages collide within 6h
suppressed -> delayed_48h            // 48h elapsed, recheck relevance
delayed_48h -> sent                   // Still relevant (festival ongoing)
delayed_48h -> expired               // No longer relevant (birthday passed)
```

Edge Cases

- Birthday on Diwali day: birthday wins (priority 1). Diwali festival P.S. appended.
- Customer in win-back returns: stamp_earned (transactional) sent immediately. Win-back auto-marked 'recovered'.
- Festival broadcast: each customer checked individually. Some may be suppressed.
- Manager manually triggers 'we miss you' to customer who just got birthday message: suppressed with toast.

Audit Actions

MESSAGE_SENT | MESSAGE_SUPPRESSED | MESSAGE_BUNDLED | MESSAGE_DELAYED |
MESSAGE_EXPIRED

Code :

6

Reward Stock-Out Behavior

Reward stock = 0. Customer wants to redeem. Alternative? Waitlist? Replace? Don't lose the moment.

Scenario

Kavya wants to redeem 'Free Brownie' but stock = 0 (Sunita forgot to bake). Kavya has a completed card. She's at the counter. Don't send her away empty-handed.

Decision Matrix

■ Block redemption — 'Sorry, out of stock'

Simplest. But customer loses trust. May not return.

■ Auto-offer cheapest in-stock alternative

Fast. But may offer a low-value reward customer doesn't want.

✓ Cashier picks from in-stock alternatives + optional waitlist for original

Flexible. Customer chooses. Waitlist captures restock intent.

RECOMMENDED POLICY

■ Auto-issue raincheck voucher (digital, 30-day validity)

Clean. But adds voucher management complexity.

Production Behavior

- DEFAULT POLICY: 'Cashier picks from in-stock alternatives + optional waitlist for original'.
- Customer at counter with completed card. Cashier taps 'Redeem' on 'Free Brownie'.
- POS shows modal: 'Free Brownie is out of stock. Options:'
 - 1. Pick alternative (list of in-stock rewards at same or lower cost)
 - 2. Waitlist for Free Brownie (card stays completed, WhatsApp when restocked)
 - 3. Save card for later (no action)
- ALTERNATIVE: standard redemption flow on the alternative reward. Original card consumed.
- WAITLIST: RewardWaitlist row created. Card stays completed. When restocked, cron checks waitlist FIFO:
 - - First customer gets WhatsApp: 'Free Brownie back in stock. Visit within 7 days.'
 - - 7-day window. If not redeemed, next customer gets the chance.
- Dashboard alert: 'Reward restock needed' when any reward stock ≤ 5 OR waitlist length > 3 .

State Machine - States

in_stock | low_stock | out_of_stock | waitlisted | restocked | notified | redeemed_via_waitlist | expired

Transitions

```
in_stock -> low_stock           // stock <= 5
low_stock -> out_of_stock        // stock = 0
out_of_stock -> waitlisted       // Customer added to waitlist
waitlisted -> restocked          // Manager updates stock > 0
restocked -> notified            // Cron notifies top N waitlist customers
```

```
notified -> redeemed_via_waitlist    // Customer redeems within 7 days
notified -> expired                    // 7 days elapse without redemption
```

Edge Cases

- All rewards out of stock: POS shows 'All rewards temporarily out of stock. Save card for later?'.
- Customer on waitlist redeems different reward: waitlist entry auto-removed.
- Reward permanently discontinued: waitlist cleared. Customers notified.
- Customer at waitlist position #5, only 3 restocked: not notified. Stays in queue.

Audit Actions

```
REDEMPTION_OUT_OF_STOCK | REDEMPTION_ALTERNATIVE_OFFERED |
REWARD_WAITLISTED | REWARD_RESTOCKED | WAITLIST_CUSTOMER_NOTIFIED
```

Code :

7

Owner Manual Override Workflow

Owner wants to manually give a customer a reward or bonus stamps. No bill, no redemption flow.

Scenario

Kavya's birthday cake was slightly burnt. Owner Ravi wants to apologize — give her 3 bonus stamps + a free pastry. Or: Meera referred 5 friends, Ravi wants to give her a surprise free cake. No bill, no redemption. Just owner discretion.

Decision Matrix

■ No override path — owner must use standard flows (Rs 0 bill)

Audit-clean. But feels hacky. Rs 0 bills pollute revenue reports.

■ Owner override: give stamps + give reward, no questions asked

Fast. But no abuse protection. Cashier could misuse if they have owner PIN.

✓ Owner override with mandatory reason + audit log + monthly cap (5/month)

Balanced. Owner has discretion but pattern is auditable.

RECOMMENDED POLICY

■ Owner override requires co-approval from manager

Strictest. But slow for time-sensitive apologies (customer at counter).

Production Behavior

- DEFAULT POLICY: 'Owner override with mandatory reason + audit log + monthly cap (5 overrides/month)'.
- Owner opens customer profile, clicks 'Manual Override'.
- Modal: 'Override type: [Bonus stamps] [Free reward] [Both]. Reason (required, min 10 chars).'
- Owner submits. System checks monthly cap.
- If cap exceeded: 'You've used 5 overrides this month. Contact support.' 403.
- If within cap: override applied.
- - Bonus stamps: Stamp rows created with source='manual_override'.
- - Free reward: Redemption row with status='completed', notes='Manual override: '. Reward.stock NOT decremented (gift).
- WhatsApp to customer (optional, owner toggles).
- Dashboard widget: 'Overrides this month: 3/5'.
- Manager CANNOT override — only Owner role.

State Machine - States

initiated | cap_checked | applied | cap_exceeded | reviewed

Transitions

initiated -> cap_checked	// Owner submits override form
cap_checked -> applied	// Monthly cap not exceeded
cap_checked -> cap_exceeded	// 5 overrides already used this month
applied -> reviewed	// Monthly support review

Edge Cases

- Owner overrides bonus stamps that complete a card: card.completed = true, reward_ready WhatsApp sent.
- Owner overrides a reward that requires manager approval: approval bypassed (owner is highest authority).
- Override used for fraud: pattern detected in monthly review. Support can suspend owner account.
- Monthly cap reset: 1st of each month. Unused cap doesn't roll over.

Audit Actions

OWNER_OVERRIDE | OWNER_OVERRIDE_CAP_EXCEEDED | OWNER_OVERRIDE_REVIEWED

Code :

8

Duplicate Customer Merge Workflow

Same customer registered twice. Merge without losing history.

Scenario

Cashier search shows 2 records: 'Meera Iyer' (5 bills, 7 stamps) and 'Meera Iyer (Bakry)' (2 bills, 4 stamps). Same phone. Different name spellings. Need to merge into one canonical record.

Decision Matrix

■ Hard delete duplicate — keep older record

Data loss. 2 bills and 4 stamps gone.

✓ Merge: copy all bills/stamps/redemptions to canonical, soft-delete duplicate

Clean. But merge direction (which is canonical?) matters.

RECOMMENDED POLICY

■ Merge: keep newer record as canonical

Risk: newer record may have less history.

■ Auto-merge on phone match (no human review)

Fast. But risky — 2 different people with same phone (family landline) wrongly merged.

Production Behavior

- DEFAULT POLICY: 'Manual merge by manager — pick canonical, migrate all data, soft-delete duplicate'.
- Manager opens Customers tab, sees duplicate flag (system auto-detects same phone).
- Clicks 'Merge' on one of the records.
- Modal: 'Select canonical record (the one to keep):' with radio buttons.
- Manager picks canonical (usually the one with more history).
- Preview: 'Record A will absorb: 2 bills, 4 stamps, 1 redemption from Record B. Record B soft-deleted.'
- Manager confirms. Transaction:
 - - All Bill/Stamp/CustomerStampCard/Redemption/Review/Referral/Achievement rows: customerId updated to canonical
 - - Customer B: status='merged', mergedIntoId=A.id, soft-deleted
 - - Customer A: lifetimeStamps += B.lifetimeStamps, lifetimeSpend += B.lifetimeSpend
 - - Birthday conflict: manager picks which to keep during merge dialog
 - - VIP tier conflict: A keeps the higher tier (never downgrade on merge)
- Merge is IRREVERSIBLE after 7 days. Within 7 days, owner can 'unmerge'.

State Machine - States

duplicate_detected | merge_initiated | canonical_selected | merged | unmerge_window | permanent

Transitions

```
duplicate_detected -> merge_initiated // Manager clicks Merge
merge_initiated -> canonical_selected // Manager picks canonical
```

```
canonical_selected -> merged           // Manager confirms preview
merged -> unmerge_window                // 7-day unmerge window starts
unmerge_window -> permanent             // 7 days elapse
unmerge_window ->
duplicate_detected                      // Owner clicks Unmerge (reverses migration)
```

Edge Cases

- 3+ duplicates (triple merge): manager merges pairwise. A absorbs B, then A absorbs C.
- Duplicate has active referral pending: referral migrates. If canonical is the referrer (self-loop), referral denied.
- Duplicate has pending redemption: migrates to canonical. Manager approves from canonical's profile.
- Family members with same phone (legitimate): merge is OPTIONAL. Manager can mark 'not a duplicate'.

Audit Actions

CUSTOMER_MERGE_INITIATED | CUSTOMER_MERGED | CUSTOMER_UNMERGED |
DUPLICATE_FLAG_DISMISSED

Code :

Merchant Ownership Transfer

Bakery sold. New owner. Staff remain. Subscription remains. Data ownership transfer flow.

Scenario

Ravi sells Sweet Crumb Bakery to Anita on Month 8. Anita wants: keep all customer data, keep staff (Priya, Sunita), keep subscription (5 months remaining), but she becomes the new Owner. Ravi should lose access.

Decision Matrix

■ New owner creates fresh account, old owner manually exports/reimports data

Data loss risk. Subscription lost. History fragmented.

✓ Ownership transfer: old owner initiates, new owner accepts, all data stays

Clean. Subscription, data, staff all preserved. Access transferred atomically.

RECOMMENDED POLICY

■ Support-mediated transfer: both owners email support

Safest but slow (2-3 business days). Bad UX for time-sensitive sales.

Production Behavior

- DEFAULT POLICY: 'Self-service ownership transfer with 7-day pending window'.
- Old owner (Ravi) opens Settings -> Ownership -> 'Transfer Ownership'.
- Form: 'New owner email + new owner name + confirmation checkbox.' + reason.
- System creates MerchantTransferRequest { oldOwnerId, newOwnerEmail, status: 'pending', expiresAt: now + 7d }.
- Email + WhatsApp to new owner (Anita): 'You've been invited to take ownership. Accept at [link]. Expires in 7 days.'
- Anita clicks link, creates account. Sees transfer preview:
 - Merchant: Sweet Crumb Bakery
 - Customers: 180 (all data preserved)
 - Staff: 2 (Priya, Sunita — will remain)
 - Subscription: 5 months remaining
- Anita clicks 'Accept Transfer'.
- Atomic transaction:
 - New Staff row: Anita, role=OWNER
 - Old owner Staff row (Ravi): status='revoked'
 - Merchant.primaryOwnerId updated to Anita.id
 - Subscription: no change (continues under merchant)
 - All other Staff: no change
 - All Customer/Bill/Stamp data: no change (tied to merchantId, not ownerId)
- Ravi gets email: 'Ownership transferred. You no longer have access.'
- ROLLBACK: within 24h of transfer, Ravi can click 'Cancel transfer'. After 24h: irreversible.
- LEGAL COMPLIANCE: transfer creates digital record (MerchantTransferLog) exportable as PDF for legal purposes.

State Machine - States

initiated | pending_acceptance | accepted | completed | expired | cancelled | rolled_back

Transitions

```
initiated -> pending_acceptance    // Old owner submits transfer form
pending_acceptance -> accepted      // New owner clicks Accept
accepted -> completed               // Atomic transfer transaction succeeds
pending_acceptance -> expired       // 7 days elapse without acceptance
pending_acceptance -> cancelled     // Old owner cancels before acceptance
completed -> rolled_back            // Old owner cancels within 24h
```

Edge Cases

- New owner already has a CustomerPilot merchant account: blocked. 'Use a different email or contact support.'
- Subscription payment failed during transfer: transfer proceeds. New owner must resolve payment.
- Old owner has pending support tickets: tickets remain open, assigned to merchant (new owner sees them).
- Old owner tries to login after transfer: 'Access revoked. Contact current owner.'
- Disputed transfer: support can reverse if AuditLog shows suspicious IP/timing. 30-day dispute window.

Audit Actions

MERCHANT_TRANSFER_INITIATED | MERCHANT_TRANSFER_ACCEPTED |
MERCHANT_OWNERSHIP_TRANSFERRED | MERCHANT_TRANSFER_EXPIRED |
MERCHANT_TRANSFER_CANCELLED | MERCHANT_TRANSFER_ROLLED_BACK |
MERCHANT_TRANSFER_DISPUTED

Code :

Scenario

Cashier Priya wants to give her friend Meera extra stamps. She creates 5 fake bills of Rs 50 each in 10 minutes (1 stamp per bill = 5 stamps). Or: cashier enters Rs 0.01 bill to award a stamp without real revenue. Or: same customer billed 20 times in one day.

Decision Matrix■ **No fraud detection — trust the cashier**

Simple. But enables stamp inflation. Owner may not notice for months.

■ **Hard rules: block any bill < Rs 10, block > 10 bills per customer per day**

Strict. But legitimate edge cases (Rs 5 sample, festival 15 bills/day) get blocked.

✓ **Soft rules: flag suspicious patterns, alert manager, don't block**

Balanced. Owner/manager reviews flags. False positives don't block legitimate sales.

RECOMMENDED POLICY■ **ML-based anomaly detection**

Most accurate. But requires training data, model maintenance. Overkill for V6.

Production Behavior

- DEFAULT POLICY: 'Soft rules — flag suspicious patterns, alert manager, don't block'.
- Detection rules (run synchronously on every POST /api/bills):
- RULE 1 — Low-value bill: amount < Rs 10 -> flag severity=low
- RULE 2 — Rapid repeat: same customerId + same amount within 5 minutes -> flag severity=high
- RULE 3 — High frequency: same customerId > 8 bills in 24h -> flag severity=medium
- RULE 4 — After-hours: bill created between 11 PM and 5 AM -> flag severity=low
- RULE 5 — Cashier self-deal: customerId matches cashier's own phone -> flag severity=high
- RULE 6 — Stamp spike: customer's lifetimeStamps increased by >10 in 24h -> flag severity=medium
- On flag: FraudAlert row created. Bill STILL proceeds (soft rule). Stamps awarded normally.
- Dashboard widget (manager+ owner): 'Fraud alerts: 3 open'.
- Manager reviews each alert: 'Mark as legitimate' or 'Investigate' (escalates to owner).
- Owner can 'Reverse stamps' on confirmed fraud: stamps deleted, card reset.
- Pattern detection: if same cashier triggers >5 fraud alerts in 30 days, auto-escalate to owner.

State Machine - States

open | reviewing | legitimate | investigating | confirmed_fraud | stamps_reversed | false_positive

Transitions

```
open -> reviewing           // Manager opens alert
reviewing -> legitimate      // Manager marks as legitimate
reviewing -> investigating  // Manager escalates to owner
investigating -> confirmed_fraud // Owner confirms fraud
```

```
investigating -> false_positive      // Owner marks as false positive  
confirmed_fraud -> stamps_reversed    // Owner reverses stamps
```

Edge Cases

- Festival rush: 15 bills/day for one customer (bulk order). Rule 3 fires. Manager marks legitimate.
- Cashier gives self a stamp (legitimate): Rule 5 fires. Manager marks legitimate.
- Multiple cashiers colluding: pattern detected across staffs. Owner alert: '3 cashiers all billing same customer 5+ times.'
- Customer legitimately visits 10 times in a day (office party): Rule 3 fires. Manager marks legitimate.
- Fraud confirmed but customer already redeemed: same pattern as refund-after-redemption. Owner absorbs cost.

Audit Actions

```
FRAUD_ALERT_CREATED | FRAUD_ALERT_REVIEWED | FRAUD_ALERT_ESCALATED |  
FRAUD_STAMPS_REVERSED | FRAUD_PATTERN_DETECTED |  
FRAUD_ALERT_FALSE_POSITIVE
```

Code :

Scenario

Ravi opens dashboard. Sees: 'Repeat Rate dropped 12% this month.' He panics. Why? Is it the new competitor? Bad cake quality? Or just seasonality? Without explanation, the number is noise.

Decision Matrix

■ **Raw numbers only — merchant interprets themselves**

Current state. Numbers without context. Merchant may misdiagnose.

■ **Static tooltips — 'Repeat Rate = customers who visited 2+ times this month'**

Helps definition. But doesn't explain the trend.

✓ **AI explanation layer — for each KPI, AI generates 'Why this changed' + 'Recommended action'**

Most useful. But requires LLM integration. Demo uses template-based.

RECOMMENDED POLICY

■ **Manual weekly report — support team emails insights**

High-quality but slow. Doesn't scale.

Production Behavior

- **DEFAULT POLICY:** 'AI explanation layer with template-based explanations (demo) -> LLM-powered (production)'.
- For each KPI on the dashboard (revenue, repeat rate, churn rate, redemption rate, reputation score):
 - - Show the number + trend arrow (up/down/flat vs last period)
 - - Show 'Why?' expandable section with AI explanation
- Explanation structure: 'Repeat Rate down 12% because: (1) 8 customers moved to dormant (vs 3 last month), (2) Birthday campaign reached fewer customers, (3) No festival campaign this month.'
- Recommended action: 'Re-engage dormant customers via win-back day_30 messages. Schedule a campaign next week.'
- Demo mode: explanations are template-based (rule engine comparing this month vs last month).
- Production mode: swap template for LLM call (z-ai-web-dev-sdk chat completion).
- Explanations generated on-demand. Cached for 1 hour per merchant.
- Merchant can give feedback: 'Helpful' / 'Not helpful' on each explanation.
- Privacy: explanations only use merchant's own data. No cross-merchant benchmarking in V6.

State Machine - States

metric_changed | explanation_requested | explanation_generated | feedback_received

Transitions

```
metric_changed -> // Merchant expands 'Why?' section
explanation_requested

explanation_requested -> // Template engine or LLM returns explanation
explanation_generated
```

explanation_generated ->
feedback_received

// Merchant clicks Helpful or Not helpful

Edge Cases

- Metric is flat: explanation = 'No significant change. Business as usual.'
- Metric is new (no last-period data): explanation = 'Insufficient historical data. Available next month.'
- Multiple factors: AI lists top 3 contributors by impact. Doesn't overwhelm with 10 reasons.
- LLM unavailable: fallback to template-based. Silent degradation.
- Contradictory metrics (revenue up but repeat rate down): AI acknowledges tension. 'Revenue up 8% (new customers) but Repeat Rate down 12% (existing not returning). Focus on retention.'

Audit Actions

AI_INSIGHT_GENERATED | AI_INSIGHT_FEEDBACK

Code :

Scenario

Sweet Crumb Bakery closes at 9 PM. But some bakeries (wedding cake specialists) take orders until 11 PM. Restaurants open until 1 AM. A 6 PM daily summary is useless for them — misses the dinner rush.

Decision Matrix■ **Fixed 6 PM daily summary for all merchants**

Simple. But useless for late-closing businesses.

■ **Fixed 11 PM daily summary**

Better. But midnight snack shops close at 3 AM.

✓ **Merchant configures business hours + timezone. Summary sent 1 hour after closing.**

Most accurate. Merchant gets summary when day is truly done.

RECOMMENDED POLICY■ **Merchant configures summary time manually**

Flexible. But merchant may pick wrong time (9 AM, missing previous evening).

Production Behavior

- **DEFAULT POLICY:** 'Merchant configures business hours + timezone. Summary sent 1 hour after closing.'
- On signup, merchant picks:
 - - Timezone (default: Asia/Kolkata for Indian merchants)
 - - Business hours: open time + close time (e.g. 9 AM - 9 PM)
 - - Closed days (e.g. Sundays closed — no summary sent)
- Default business hours by type:
 - - cafe: 8 AM - 8 PM
 - - bakery: 9 AM - 9 PM
 - - salon: 10 AM - 7 PM
 - - restaurant: 11 AM - 11 PM
 - - retail: 10 AM - 10 PM
- **DAILY SUMMARY CRON:** runs every 15 minutes. For each merchant:
 - - Compute current time in merchant's timezone
 - - If current time == closing time + 1 hour AND today is not a closed day:
 - - Generate daily summary WhatsApp
- **EDGE CASE — past-midnight closing** (restaurant closes at 1 AM):
 - - Summary sent at 2 AM. Counts as 'yesterday's' summary.
 - - Bills from 12 AM - 1 AM counted in previous day's summary.
- **EDGE CASE — 24-hour business:** summary sent at 3 AM (quietest hour).
- **EDGE CASE — multi-shift business:** single daily summary at 3 AM.
- **TODAY'S BRIEFING** (morning): sent at opening time.
- Merchant can override: 'Send summary at fixed X PM regardless'.

State Machine - States

before_close | closing_window | summary_window | summary_sent | closed_day

Transitions

```
before_close -> closing_window      // Current time in closing hour
closing_window -> summary_window     // Closing time reached
summary_window -> summary_sent       // 1 hour after closing (cron fires)
summary_sent -> before_close         // Next day's opening time
before_close -> closed_day           // Today is a closed day
closed_day -> before_close           // Next day (open day)
```

Edge Cases

- Daylight saving: India doesn't observe DST, but other countries might. Timezone-aware cron handles automatically.
- Merchant changes business hours mid-month: cron picks up new hours immediately. No historical regeneration.
- Merchant closes early unexpectedly (sick day): no summary sent that day. Next day's summary covers 2 days.
- Internet outage at summary time: cron retries every 15 min for 2 hours. If still failing, summary skipped.
- Merchant in different timezone than customers: summary always in merchant's timezone.

Audit Actions

DAILY_SUMMARY_SENT | DAILY_SUMMARY_SKIPPED_CLOSED_DAY |
DAILY_SUMMARY_FAILED | MORNING_BRIEFING_SENT | BUSINESS_HOURS_CHANGED

Code :